

Cumulative Sum/Running Total

One of the **most asked questions** in the world of data is how to calculate **cumulative sum** (a.k.a. **running total**). Whether you're working in a relational database (RDBMS) or a big data framework like PySpark, this is a must-know skill for any data professional.

simple examples:

In RDBMS (e.g., Oracle SQL)

Imagine we have a sales_data table:

SALE_ID	SALE_DATE	AMOUNT
1	2025-01-01	100
2	2025-01-02	150
3	2025-01-03	200

To calculate the **running total** for amount:

```
SELECT
    sale_id,
    sale_date,
    amount,
    SUM(amount) OVER (ORDER BY sale_date) AS running_total
FROM
    sales_data;
```

Output:

SALE_ID	SALE_DATE	AMOUNT	RUNNING_TOTAL
1	2025-01-01	100	100
2	2025-01-02	150	250
3	2025-01-03	200	450

In PySpark

PySpark is widely used for big data processing, and calculating a **cumulative sum** is just as easy!

Here's how it works:

```

from pyspark.sql import SparkSession

from pyspark.sql.window import Window

from pyspark.sql.functions import col, sum

# Create Spark session

spark = SparkSession.builder.appName("CumulativeSum").getOrCreate()

# Sample data

data = [(1, "2025-01-01", 100), (2, "2025-01-02", 150), (3, "2025-01-03", 200)]

columns = ["sale_id", "sale_date", "amount"]

# Create DataFrame

df = spark.createDataFrame(data, columns)

# Define a window

windowSpec = Window.orderBy("sale_date")

# Add cumulative sum column

df = df.withColumn("running_total", sum("amount").over(windowSpec))

df.show()

```

Output:

SALE_ID	SALE_DATE	AMOUNT	RUNNING_TOTAL
1	2025-01-01	100	100
2	2025-01-02	150	250
3	2025-01-03	200	450

 Key Takeaways:

- **RDBMS:** Use SUM() with OVER() to calculate running totals directly in SQL.
- **PySpark:** Use the Window function with sum() for scalable cumulative calculations.